Compilation Process & Stages

Advanced Toolchain & File Handling

1. What is a build system and why is it important for compilation?
    a. A build system automates the process of compiling source code, linking, and generating executables or libraries. It manages dependencies and only recompiles files that have changed, saving time and avoiding redundant work.
    b. Example: make is a popular build system using Makefile scripts to define build rules and dependencies.
2. How do static and dynamic linking differ?
    a. Static linking: All necessary library code is copied into the final executable during linking. Resulting binary is self-contained but larger.
    b. Dynamic linking: Libraries are loaded at runtime. Executable contains references but not the library code, resulting in smaller files and sharing libraries across programs.
    c. Example: Windows .exe often uses dynamic linking for DLLs; embedded systems typically prefer static linking.
3. What is a linking error and how can it be resolved?
    a. Linking errors occur when the linker can't find references to external symbols (functions or variables).
    b. Example: Missing definition for a function declared in a header causes: undefined reference to 'myFunc'
    c. Resolved by ensuring the object file or library implementing the symbol is included in the link command.
4. How do linker scripts work, and how do they help place code and data in memory regions?
    a. Linker scripts define memory layout and control placement of code and data sections.
    b. They assign sections like .text, .data, .bss to specific addresses or memory banks.
    c. Crucial for embedded to optimize limited memory and define vector tables, RAM, and peripheral mappings.
5. What is a symbol table and where is it stored?
    a. A symbol table stores names of variables/functions with memory addresses and scope info, typically embedded in object or executable files to enable linking and debugging.
6. Can object files contain debug information, and how is it utilized?
    a. Yes, object files can include debug symbols mapping compiled code to source code lines, used by debuggers (e.g., GDB) for source-level debugging.

7. What are ELF sections .text, .data, and .bss used for?
    a. .text: Executable code
    b. .data: Initialized global/static variables
    c. .bss: Uninitialized global / static variables (initialized to zero at runtime)

Cross-Compilation, Firmware, and Embedded Details

1. What are bootloaders, and how are they related to .bin/.hex files?
    a. A bootloader is a small program loaded at startup that initializes hardware and loads main firmware.
        i. Bootloader code and main firmware are often stored as .bin or .hex files for flashing.
        ii. The bootloader may load additional .bin or .hex firmware files from storage or over network.
2. What's the role of a startup file in microcontroller programming?
    a. Startup files set up the initial stack, initialize memory and hardware, define interrupt vectors, and jump to main().
        i. Written in assembly or C.
        ii. Critical for proper system initialization before main firmware executes.
3. Why is binary size optimization critical in embedded development?
    a. Embedded devices have limited flash and RAM.
    b. Smaller binaries reduce memory and power consumption.
    c. Allows fitting more features or complex logic within constraints.
    d. Techniques: function inlining, dead code elimination, link-time optimization.
4. How can conditional preprocessing directives impact firmware builds?
    a. Enable/disable features or debugging code.
    b. Produce different firmware variants from the same source.
    c. Manage hardware differences across target boards.
    d. Example:

```
#ifdef DEBUG
        printf("Debug info\n");
#endif
```